PROJECT DOCUMENTATION

AI FitTrack API

FitSense: Personalized fitness recommendations powered by AI

A secure REST backend for workout tracking and AI-assisted fitness guidance

TECHNOLOGY STACK	Node.js | Express.js | MongoDB | Google Gemini AI
DOCUMENT TYPE	Project Documentation
VERSION	1.0
DATE	September 2026

Contents

1	Introduction and Purpose	3
2	Key Features and Architecture	4
3	Requirements	5
4	Setup and Running the API	6
5	Data Model and API Endpoints	7
6	API Testing	8
7	Future Scope	9
8	Conclusion	10

About this document

This document gives a simple, high-level overview of the AI FitTrack API. It explains what the API is for, what is needed to build and run it, how it was tested, and where it can grow next. It is meant as a quick guide, not a full technical manual.

> **In one line:** AI FitTrack is a secure backend that lets users register, log and search their workouts, and receive personalized fitness recommendations and insights from Google Gemini AI.

1 Introduction and Purpose

Many people who exercise regularly still record their workouts by hand in notebooks or spreadsheets. This makes history hard to search, progress hard to see, and calculations easy to get wrong. It also gives no guidance on which workouts suit a person's goals.

AI FitTrack solves this with a central backend service. Users sign up securely, save and manage their workouts, and ask the built-in AI for advice.

Main purpose of the API

- **Track workouts** in one secure, central place instead of manual notes.
- **Protect personal data** using login, password hashing and token-based access.
- **Find past workouts** quickly by name, category or date.
- **Guide users with AI** by generating workout plans and fitness insights.

Problems it addresses

Problem today	How AI FitTrack helps
Workout history scattered across notes and sheets	One central record per user
No secure access to personal fitness data	JWT login with hashed passwords
Hard to search old workouts	Search by name, category or date
No personalized guidance	AI workout recommendations
No view of overall performance	AI fitness insights from workout statistics

Who uses it

The API has one role: the **authenticated user**. Any web, mobile or third-party app can connect to it, because every response is returned as standard JSON.

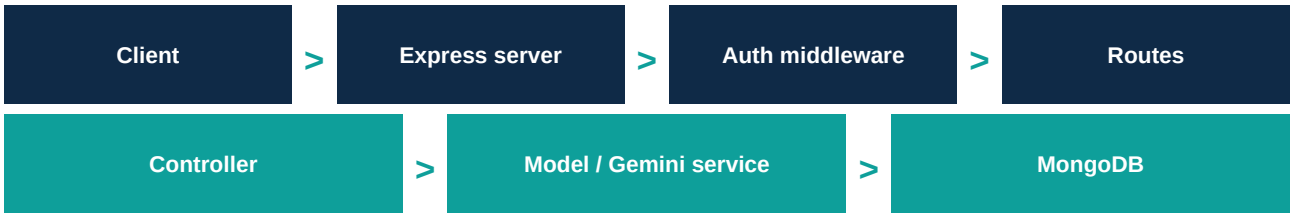
2 Key Features and Architecture

The API is built on the Model-View-Controller (MVC) pattern, so each part of the code has one clear job. This keeps the project easy to maintain and extend.

Key features

Feature	What it does
Secure authentication	Register, log in, view profile; passwords hashed with bcrypt; access via JWT
Workout management	Add, view, update and delete workouts, each linked to its owner
Workout search	Filter history by workout name, category or date
AI recommendations	Personalized plan from age, fitness goal and experience level
AI fitness insights	Feedback from total workouts, average duration and calories burned
Error handling	Clear JSON messages for invalid logins, missing tokens and bad input

How a request flows



A request enters the Express server, is checked by the JWT middleware, and is passed by the routes to a controller. The controller uses the Mongoose models for database work or the Gemini service for AI work, then returns a JSON response.

3 Requirements

What is needed to create and run the API.

Software

Component	Purpose
Operating system	Windows 10/11, macOS or Linux
Node.js (v16 or above)	Runs the server code
npm (v8 or above)	Installs and manages packages
MongoDB	Stores users and workouts
Postman or Thunder Client	Sends requests to test the API
VS Code (or similar)	Code editor

Hardware

Item	Minimum
Processor	Intel Core i5 (8th gen) / AMD Ryzen 5 or equivalent
RAM	8 GB (16 GB recommended)
Storage	1 GB free disk space

Packages and services

Package / service	Role
express	Web framework and routing
mongoose	MongoDB models and validation
dotenv	Loads environment variables
bcryptjs	Password hashing
jsonwebtoken	Creates and verifies JWT tokens
cors	Allows requests from other origins
nodemon (dev)	Restarts the server on code changes
Google Gemini API key	Needed for the AI features

4 Setup and Running the API

A short walkthrough from empty folder to running server.

Setup steps

- Create a project folder (for example **AI-FitTrack-API**) and open it in VS Code.
- Create a **server** folder for the backend code and open a terminal inside it.
- Initialize the project and install the packages:

```
npm init -y
npm install express mongoose dotenv bcryptjs jsonwebtoken cors
npm install --save-dev nodemon
```

Environment variables

Sensitive values live in a **.env** file so they never appear in the source code.

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/aifittrack
JWT_SECRET=your_secret_key
GEMINI_API_KEY=your_google_gemini_api_key
```

Project structure

Folder	Contents
config/	Database connection
models/	User and Workout schemas
routes/	Auth, workout and AI routes
controllers/	Request handling and business logic
middleware/	JWT authentication and error handler
services/	Gemini AI service
app.js / server.js	Server entry point

Running

Make sure MongoDB is running, then start the server with `npm start`. The API is then available at `http://localhost:5000`.

5 Data Model and API Endpoints

The database has two collections. One user can own many workouts (one-to-many), linked through the **user** field in each workout.

User	Workout
name, email (unique), password (hashed), timestamps	workoutName, category, duration (min), caloriesBurned, workoutDate, user, timestamps

Supported workout categories: Cardio, Strength Training, Yoga, Running, Cycling and Walking.

Endpoint overview

Method	Endpoint	Description	Auth
POST	`/api/auth/register`	Create an account and receive a token	No
POST	`/api/auth/login`	Log in and receive a token	No
GET	`/api/auth/profile`	View the logged-in user's profile	Yes
POST	`/api/workouts`	Add a workout	Yes
GET	`/api/workouts`	List all of the user's workouts	Yes
GET	`/api/workouts/:id`	Get one workout by ID	Yes
PUT	`/api/workouts/:id`	Update a workout	Yes
DELETE	`/api/workouts/:id`	Delete a workout	Yes
POST	`/api/ai/workout-recommendation`	AI workout plan from age, goal, experience	Yes
POST	`/api/ai/fitness-insights`	AI insights from workout statistics	Yes

Workout search by name, category or date is also available. Protected endpoints need the header `Authorization: Bearer <token>`.

6 API Testing

Tested manually with Thunder Client (Postman can be used the same way).

Testing followed the order in which a user would use the API: authentication first, then workouts, then AI. After registering or logging in, the returned token was added as a Bearer token for all protected requests.

Test results

Area	Test	Expected result	Result
Auth	Register user	201 Created, token and user returned	**Passed**
Auth	Login	200 OK, new token returned	**Passed**
Auth	Get profile with token	200 OK, user details returned	**Passed**
Workouts	Add workout	201 Created, workout saved with user ID	**Passed**
Workouts	Get all workouts	200 OK, list with count	**Passed**
Workouts	Get workout by ID	200 OK, single workout	**Passed**
Workouts	Update workout	200 OK, duration and calories updated	**Passed**
Workouts	Delete workout	200 OK, removal message	**Passed**
AI	Workout recommendation	200 OK, personalized plan text	**Passed**
AI	Fitness insights	200 OK, insight text	**Passed**

Sample request: AI recommendation

```
POST /api/ai/workout-recommendation
{ "age": 22, "fitnessGoal": "Weight Loss", "experience": "Beginner" }
```

The API replied with a beginner-friendly plan: three sessions per week, a short warm-up, light cardio, bodyweight strength exercises and a cool-down. AI responses took a few seconds, while normal workout requests returned in well under a second.

7 Future Scope

The modular design lets new features be added without major changes.

Enhancement	Benefit
Nutrition tracking	Log meals and connect diet to workout goals
BMI calculation	Simple health indicator from height and weight
Progress tracking	Track weight and body-fat trends over time
Wearable integration	Import steps, heart rate and calories automatically
Workout reminders	Notifications to help users stay consistent
Analytics dashboard	Charts of workout history and performance
Frontend apps	React web app and mobile app using this API
Richer AI features	Plans that adapt to a user's saved workout history

Improvements to the API itself

- Input validation for all endpoints and rate limiting for the AI routes.
- Automated tests so changes can be checked quickly.
- Pagination for long workout histories.
- Deployment to a cloud host with a managed MongoDB database.

> **Direction:** the current version is a solid foundation. The next steps are to add a user-facing app and to make the AI advice more personal by using each user's own workout data.

8 Conclusion

AI FitTrack replaces manual workout notes with a secure, organized backend. Users can register, manage and search their workouts, and get AI-generated recommendations and insights, all through a clean REST API.

The layered MVC design keeps the code simple to understand, and testing confirmed that authentication, workout management and AI features all work as expected.

Summary

Topic	Summary
Purpose	Secure workout tracking with AI guidance
Technology	Node.js, Express.js, MongoDB, Mongoose, Google Gemini
Security	bcrypt password hashing, JWT-protected routes
Data	Two collections: Users and Workouts (one-to-many)
Testing	Auth, workout and AI endpoints all passed
Next steps	Nutrition, wearables, reminders, dashboards

References

- Node.js and Express.js documentation
- MongoDB and Mongoose documentation
- JSON Web Token (JWT) standard
- Google Gemini API documentation